

Relatório BII

Guilherme Guerreiro e Miguel Alvito

2 de setembro de 2026

Resumo

Trabalhos recentes propuseram novas variantes de Random Access List, a Improved e Advanced Myers List, que reduziram o número de travessias no pior caso, no entanto, a variante Advanced fá-lo à custa da perda da operação cons em tempo constante. Este relatório levanta duas questões (i) é possível recuperar o cons em tempo constante na Advanced Myers List removendo as restrições impostas? (ii) é possível aplicar as ideias centrais da Advanced Myers List à Improved Myers List para melhorar o seu desempenho de lookup sem perder o cons $O(1)$? Para responder, propomos duas novas estruturas a Dense Advanced Myers List e a Improved Applicative Random-Access Stack with Sigma region (IARASS). Avaliamos as diferentes variantes em número de travessias, tempo de execução e uso de memória para listas de até 1.000.000 de elementos. Os resultados mostram que a variante Dense Advanced Myers List obtém o melhor desempenho de lookup ao custo do maior uso de memória, enquanto a IARASS apresenta um equilíbrio entre desempenho e memória quando comparada com a Advanced Myers List.



TRUST LAB
UNIVERSIDADE DO ALGARVE

Índice

1	Introdução	3
2	Enquadramento Teórico	3
2.1	Basic Myers List	3
2.2	Improved Myers List	4
2.2.1	Construção e conceito de região	4
2.2.2	<i>Lookup</i>	5
2.3	Advanced Myers List	5
2.3.1	Tipos de Nós	5
2.3.2	Construção	6
2.3.3	<i>Lookup</i>	6
3	Estruturas Propostas	7
3.1	Dense Advanced Myers List	7
3.1.1	<i>Cons</i>	7
3.1.2	<i>Lookup</i>	8
3.2	Improved Applicative Random-Access Stack with Sigma region	8
3.2.1	<i>Cons</i>	8
3.2.2	<i>Lookup</i>	9
4	Resultados	9
4.1	Número de Travessias	9
4.2	Tempo de Execução	11
4.3	Uso de Memória	12
5	Discussão	12
6	Conclusão	13

1 Introdução

A lista ligada, também conhecida por *cons list*, é um pilar da programação funcional e uma das estruturas de dados mais simples e elegantes. Como as listas ligadas são apenas células ligadas sequencialmente por apontadores, operações como *cons* (adicionar um elemento ao início), *car* (aceder ao primeiro elemento) e *cdr* (aceder à lista excluindo o primeiro elemento) apenas requerem tempo e espaço $\mathcal{O}(1)$. No entanto, aceder a uma célula arbitrária da lista exige percorrer até $n - 1$ apontadores para uma lista de comprimento n .

Para resolver este problema, Eugene W. Myers [1] propôs uma nova estrutura de dados, a Applicative Random-Access Stack, que denominamos Myers List. Essa estrutura mantém as operações de *cons*, *car* e *cdr* em tempo constante, permitindo ainda reduzir a complexidade de *lookup* para uma constante $\mathcal{O}(\log n)$, para ser mais preciso $3 \log(n) - 5$ apontadores. Esta melhoria é possível observar pela adição de um apontador adicional, denominado *jump*, em cada célula.

Trabalhos mais recentes, como o de Edward Peters, Yong Qi Foo e Michael D. Adams [3], aprimoraram a construção e o uso desse apontador *jump*, resultando nas variantes apresentadas no artigo, Improved Myers List e Advanced Myers List. Essas estruturas reduziram ainda mais o número de travessias necessárias no pior caso, obtendo uma complexidade de $2 \log(n + 1) - 3$ e $(1 + \frac{1}{\sigma}) \log(n) + \sigma + 9$ respetivamente, aproximando-se do limite teórico da informação: $\log(n + 1) - 1$.

Apesar dessas melhorias, a Advanced Myers List apresenta uma limitação, a perda da operação *cons* em tempo constante, qual acreditamos ser de extrema importância. Além disso, durante todo o estudo do artigo referenciado, os autores propõem algumas restrições, sendo uma delas a evitar adicionar novos apontadores em cada célula. Isso motiva as duas perguntas centrais deste trabalho: **(i)** é possível recuperar a operação *cons* em tempo constante na Advanced Myers List, removendo a restrição do projeto, ou seja, permitindo a adição de apontadores adicionais por célula enquanto se preserva a lógica e a estrutura apresentada no artigo? **(ii)** removendo novamente a mesma restrição, é possível aplicar as ideias centrais propostas na Advanced Myers List à Improved Myers List, de modo a melhorar o seu desempenho de *lookup* e mantendo o *cons* constante?

Neste relatório, propomos duas novas estruturas de dados, as suas respetivas implementações e bibliotecas, que respondem a estas duas perguntas: a Dense Advanced Myers List (Secção 3.1) e a Improved Applicative Random-Access Stack with Sigma region (IARASS) (Secção 3.2). Na secção 4 apresentamos os resultados experimentais de desempenho e de uso de memória. Na secção 5 discutimos as implementações destes resultados face às perguntas colocadas.

2 Enquadramento Teórico

2.1 Basic Myers List

A Myers List original, proposta por Eugene W. Myers (1983) [1], introduz um apontador *jump* adicional em cada célula de uma *cons list*, permitindo *lookup* em $3 \log(n) - 5$ apontadores no pior caso. A sua construção iterativa preserva o suporte a *cons* em tempo $\mathcal{O}(1)$, mas o seu desempenho assintótico fica aquém do limite teórico de informação.

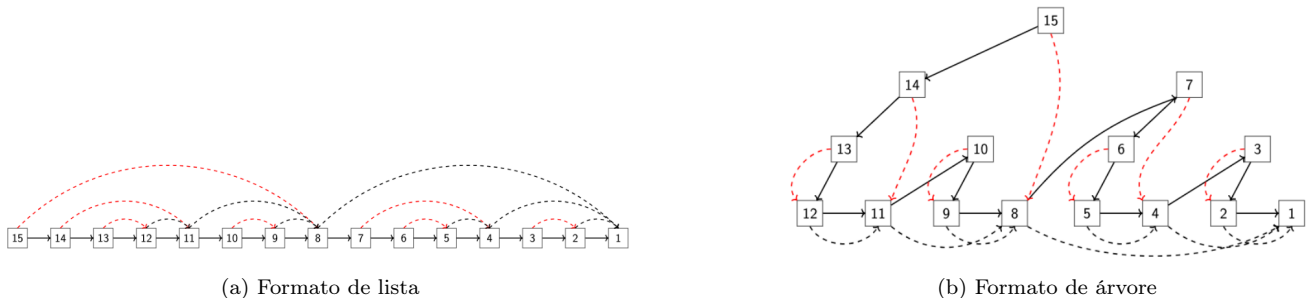


Figura 1: Representação da Basic Myers List

Assumindo as seguintes células, a associação do apontador `jump` para uma célula nova c funciona da seguinte forma:

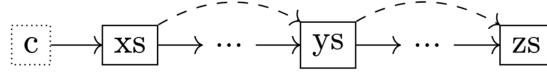


Figura 2: Exemplo de Basic Myers List, como células nomeadas

$$c.\text{jump} = \begin{cases} \text{zs} & \text{if } \text{xs.length} - \text{ys.length} = \text{ys.length} - \text{zs.length} \\ \text{xs} & \text{otherwise} \end{cases}$$

Figura 3: `jump`

Lookup na Basic Myers List é bastante simples, assume-se que `len` é um valor válido:

$$\text{lookup } \text{xs } \text{len} = \begin{cases} \text{xs} & \text{if } \text{xs.length} = \text{len} \\ \text{lookup } \text{xs.jump } \text{len} & \text{if } \text{xs.jump.length} \geq \text{len} \\ \text{lookup } \text{xs.next } \text{len} & \text{otherwise} \end{cases}$$

Figura 4: *Lookup*

2.2 Improved Myers List

A Improved Myers List, proposta por Peters, Foo e Adams (2025) [3], introduz uma modificação simples, mas eficaz à maneira como a Basic Myers List é construída. O apontador `jump` da primeira célula de cada região passa a apontar para a primeira célula da segunda sub-região, em vez da última célula. Assim, o resultado é uma melhoria do pior caso de *lookup* de $3 \log(n) - 5$ para $2 \log(n + 1) - 3$ travessias, igualando ao desempenho da Okasaki List, estrutura de dados puramente funcional proposta por Chris Okasaki [2], mas ainda com a vantagem de que permite o *lookup* a partir de qualquer posição.

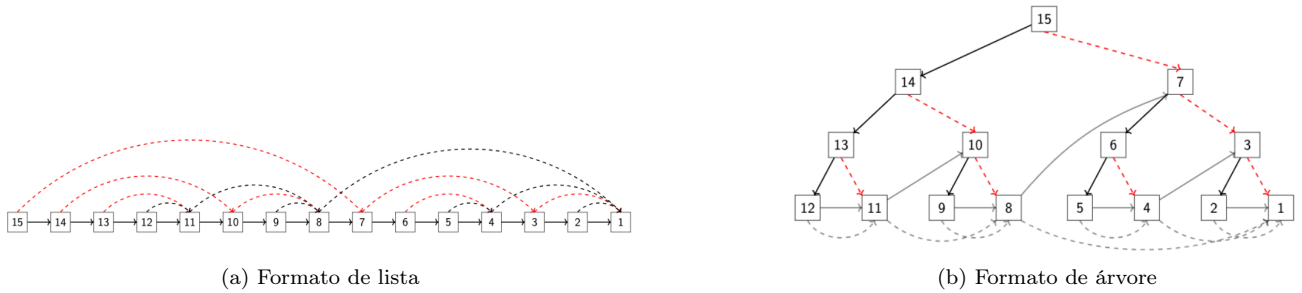


Figura 5: Representação da Improved Myers List

2.2.1 Construção e conceito de região

Uma região é definida pelo seu grau k :

- Uma região de grau $k = 0$ consiste numa única célula.
- Uma região de grau $k > 0$ é composta por duas sub-regiões de grau $k - 1$ (no artigo, estas regiões são denominadas Q_{k-1} e R_{k-1}) e uma célula adicional no início.

Na Basic Myers List, o apontador *jump* da célula inicial de uma região aponta para a última célula dessa mesma região. Adicionalmente, as duas sub-regiões partilham células, o que torna a estrutura menos eficiente. A Improved Myers List resolve este problema alterando a regra de construção: as sub-regiões Q_{k-1} e R_{k-1} deixam de partilhar células, e o apontador *jump* da célula inicial passa a apontar diretamente para a primeira célula de R_{k-1} . Se visualizarmos a lista como uma árvore, as sub-regiões Q_{k-1} e R_{k-1} formam, respetivamente, as sub-árvores esquerda e direita. Assim, o acesso à raiz da sub-árvore direita passa a custar apenas a travessia de um único apontador (*jump*), eliminando a ineficiência da variante clássica.

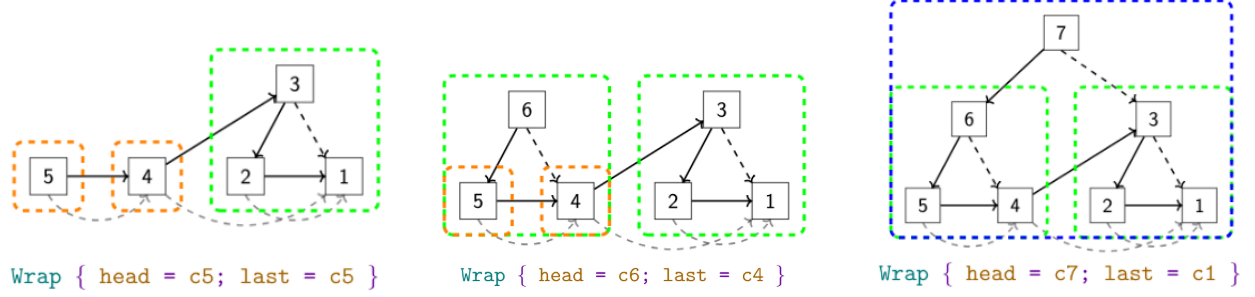


Figura 6: Evolução do wrap durante o *cons* na Improved Myers List

2.2.2 Lookup

O algoritmo de *lookup* na Improved Myers List é idêntico ao da Basic Myers List, percorre *jump* se *xs.jump.length* $\geq$ *len*, caso contrário, percorre *next*.

2.3 Advanced Myers List

A Advanced Myers List, introduzida por Peters [3], parte da Improved Myers List e introduz apontadores adicionais (*uncle*, *inc*, *greater*), permitindo assim um *lookup* que pode ser realizado em tempo aproximadamente logarítmico $(1 + \frac{1}{\sigma}) \log(n) + 9 + \sigma$ à custa de maior complexidade estrutural e de construção.

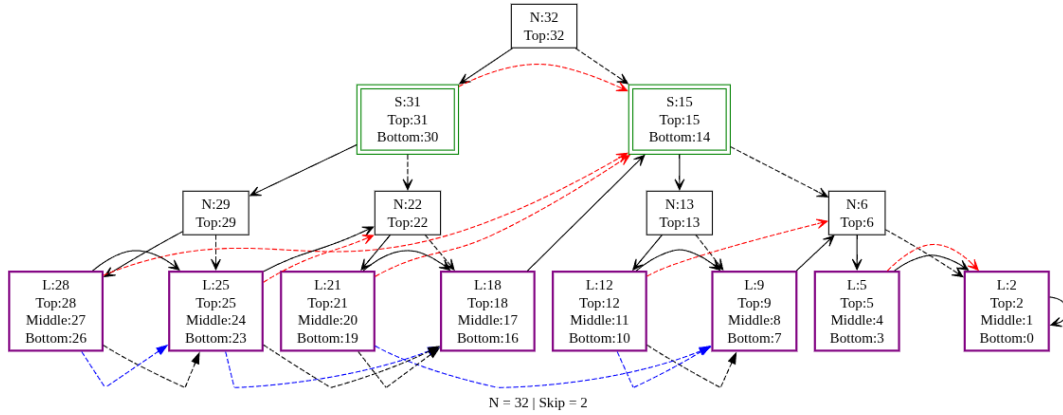


Figura 7: Advanced Myers List

2.3.1 Tipos de Nós

A Advanced Myers List organiza as células em o que é chamado de nós lógicos, estes consistem em uma célula sozinha ou a junção de duas ou três células. Existem três tipos de nós:

- Nós normais – são constituídos por apenas uma única célula (*top*), contendo apontadores *next* e *jump* a apontar diretamente para os filhos esquerdo e direito.
- Nós *skip* – são constituídos por duas células, denominadas *top* e *bottom*. O apontador *next* da célula *top* liga diretamente à célula *bottom* e o apontador *jump* aponta para o *uncle*. Estes nós aparecem repetidamente em camadas específicas, mais concretamente a cada intervalo σ acima da camada folha.

- Nós folha – São constituídos pela junção de três células: *top*, *middle* e *bottom*. Os apontadores *jump* de cada célula são reaproveitados para atalhos distintos: *inc*, *greater* e *uncle*. Estes nós constituem inteiramente a camada inferior da árvore lógica.

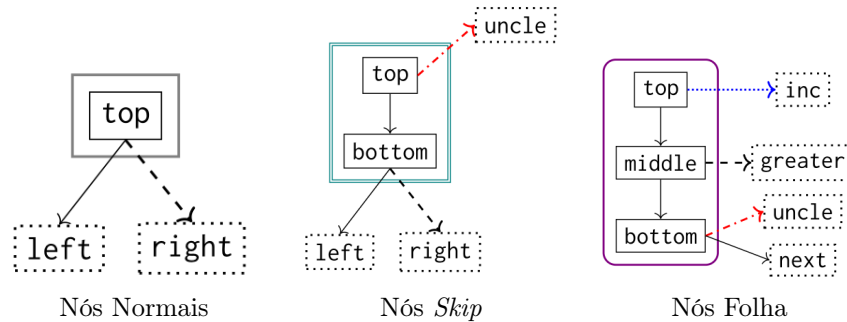


Figura 8: Tipos de Nós

2.3.2 Construção

Antes de discutir a construção é importante primeiro compreender os seguintes conceitos:

- *Right-Handed Depth* (RHD) – Corresponde à distância do ancestral mais próximo que é um filho esquerdo.
- *Right-Edge Depth* (RED) – Corresponde à distância ao ancestral mais próximo que se encontra no caminho mais à direita a partir da raiz da árvore.
- *Target uncle* – Durante o algoritmo de *lookup*, o *target uncle* refere-se à criança direita do nó ancestral comum entre o nó de origem e o nó de destino.

A construção da Advanced Myers List perde a função de cons em tempo constante, pois exige saber previamente o número exato de elementos para calcular a altura da árvore lógica e formar uma árvore binária perfeita, utilizando elementos que servem como enchimento (*fillers*) caso necessário. A estrutura é construída recursivamente do head até a camada de folhas e a atribuição dos apontadores depende do tipo de nó:

- Nós normais e *skip*: O apontador *next* aponta para a sub-árvore esquerda e o *jump* para a sub-árvore direita
- Nós *skip* e folhas: O apontador *uncle* liga a um tio onde a regra $RED(s.uncle) = RHD(s)$ é verificada
- Nós folha: O apontador *inc* liga a uma folha mais próxima à direita tal que $RHD(f') = RHD(f) + 1$. O apontador *greater* liga à folha mais próxima à direita onde $RHD(f') > RHD(f)$, este apontador pode ser obtido recorrendo ao processo usado para o apontador *jump*.

2.3.3 Lookup

O algoritmo de *lookup* na Advanced Myers List determina dinamicamente o percurso ideal com base na posição relativa entre a célula de origem e a de destino, ramificando-se em três cenários fundamentais:

- Descida Direta: Se o destino é um descendente direto da origem na árvore lógica, o algoritmo realiza uma descida através dos apontadores até alcançar o índice pretendido;
- Salto via *uncle*: Se existir um nó *skip* ou folha descendente da origem cujo RHD coincida com o RED do *target uncle*, o algoritmo salta diretamente para o *target uncle* através do apontador *uncle*;
- Travessia Lateral: Caso as condições de descendência não se apliquem, o algoritmo recorre aos apontadores *inc* para navegar horizontalmente entre nós folha adjacentes, ou ao apontador *greater* para saltar para a extremidade da subárvore corrente antes de transitar para a região seguinte da lista;
- Transição Ascendente: Quando nenhum dos três cenários anteriores é aplicável (por exemplo, quando a origem é uma folha cujo RHD já excede o RED necessário para o *target uncle*), o algoritmo segue o apontador *next* para avançar para um nó não-folha. A partir dessa nova posição, é garantido que um dos três primeiros cenários será acionado.

NOTA: Aqui explicamos o funcionamento da Improved Myers List e da Advanced Myers List de uma forma muito superficial mas rápida de compreender, para uma compreensão e para um estudo mais informativo recomenda-se ler o artigo original.

3 Estruturas Propostas

3.1 Dense Advanced Myers List

A Dense Advanced trata-se de uma variação da Advanced Myers List criada por Peters [3], mas em vez de dividir um nó com dois a quatro apontadores, em um a três células, cada uma com dois apontadores. Decidimos representar cada nó com uma única célula, com o seu respetivo número de apontadores:

```
type 'a add_points =
| Normal
| Skip of 'a cell
| Leaf of 'a cell * 'a cell
and 'a cell = { next : 'a cell; jump : 'a cell; length : int;
value : 'a; rhd : int; red : int; more : 'a add_points }
```

Listing 1: Tipos de células

Isto tornou a implementação da estrutura mais fácil e por consequência, determinar o algoritmo para a operação *cons* foi mais simples. Outros benefícios desta alteração foram: Um número de travessias de apontadores menor; Encontraram-se novas otimizações que não foram ponderadas na implementação original; E obteve-se um *lookup* mais rápido.

No entanto, um defeito desta estrutura em relação à original, é que esta gasta mais memória, já que cada célula contém mais apontadores.

O caso uso mais apropriado para esta estrutura seria um ambiente onde a travessia de apontadores seria muito custosa, tal como referido para a lista original: «Thus, advanced Myers lists may not be appropriate for in-memory lists, but they may be useful for applications where the cost of pointer traversals is high relative to other computations, such as when pointer traversals involve network access.» (Peters et al., 2025, p. 13)

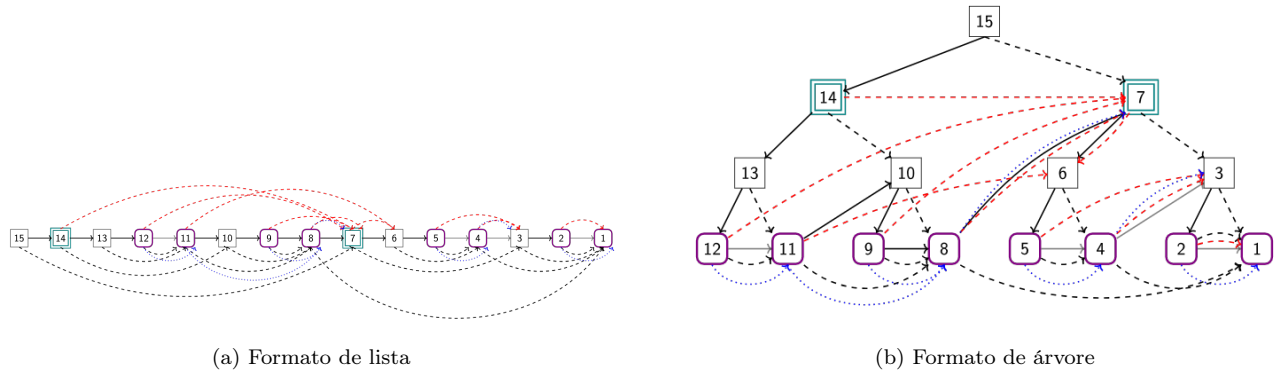


Figura 9: Representação da Dense Advanced Myers List

3.1.1 Cons

Na Dense Advanced Myers List, o *cons* difere das restantes variantes, uma vez que este não apresenta um *cons* em tempo constante. A inserção de um novo elemento exige sempre o cálculo do RED e do RHD, além da atribuição dos apontadores *next* e *jump*.

O algoritmo determina o tipo de nó a ser criado com base na altura lógica e avaliando em relação ao σ . O motivo desta estrutura não apresentar um *cons* em tempo constante deve-se a criação de novas células *skip* (Skip) e folha (Leaf), estas células apresentam apontadores adicionais: *inc* e *uncle*. Que para receberem o valor apropriado

é necessário atravessar a própria estrutura da lista à procura das células apropriadas com base nos valores do RED e RHD.

3.1.2 Lookup

O algoritmo de *lookup* na Dense Myers List é como uma busca hierárquica. Quando solicitamos o *lookup* por um elemento em uma posição específica, o algoritmo primeiro calcula a distância que precisa para chegar ao alvo e avalia o tipo de nó atual, que pode ser uma célula do tipo normal, *skip* ou folha. Se o destino estiver longe, o algoritmo usa os apontadores adicionais, *uncle* e *inc*. À medida que o algoritmo se aproxima do local desejado e a distância diminui, ele deixa de usar esses atalhos grandes e entra em uma etapa de descida. Nessa etapa final, ele faz saltos menores, *next* ou *jump*, até chegar com exatidão ao elemento procurado.

3.2 Improved Applicative Random-Access Stack with Sigma region

A Improved Applicative Random-Access Stack with Sigma region (IARASS) é uma variante da Improved Myers List que se beneficia da introdução do conceito de *sigma layers*, introduzido por Peters [3] na Advanced Myers List. Assim, o objetivo da IARASS é utilizar este novo apontador para aliviar o problema da descida dupla (*double descent*) identificado no artigo original, sem perder a simplicidade estrutural existente na variante Improved. Na secção dos Resultados, mostraremos ainda que esta estrutura, como é de esperado, apresenta um maior uso de memória que a Improved Myers List, no entanto apresenta um menor uso de memória quando comparada com a Advanced Myers List e apresentando um maior desempenho no número de travessias quando comparada com ambas as variantes.

Ao contrário da Improved Myers List em que todas as células apresentam a mesma estrutura, contendo os campos *next*, *jump*, *length* e *value*, a IARASS distingue dois tipos de células:

```
type 'a cell =
| Normal of {next:'a cell; jump:'a cell; length:int; value:'a; height:int }
| Skip of {next:'a cell; jump:'a cell; length:int; value:'a; height:int; shortcut:'a cell }
```

Listing 2: Tipos de células

Nesta variante, as células normais mantêm apenas os dois apontadores já existente na variante Improved Myers List mantendo toda a lógica já existente, enquanto as células *skip* (células de salto) apresentam um terceiro apontador, denominado *shortcut*. A decisão de qual célula é criada é baseada na altura (*height*).

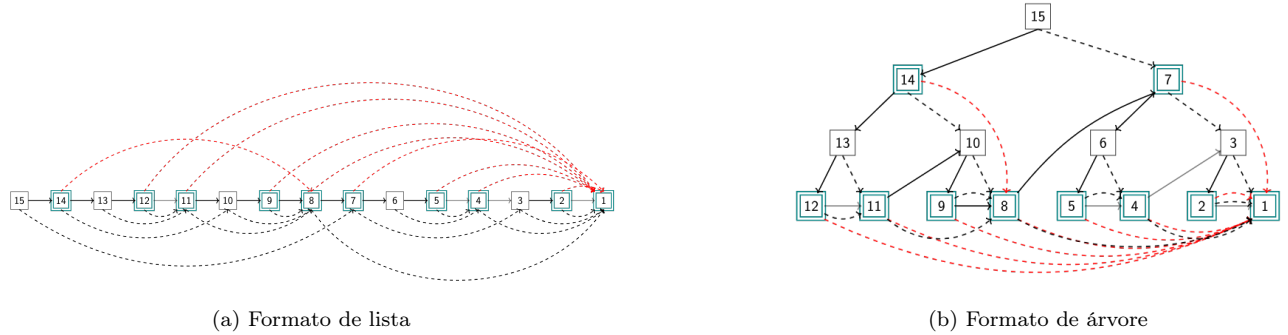


Figura 10: Representação da IARASS

3.2.1 Cons

Durante a construção de uma nova célula, sempre que a altura é um divisor de σ , ou seja $\text{height} \bmod \sigma = 0$, a célula criada é do tipo *skip* e nas restantes camadas serão do tipo normal. Ao contrário da Advanced Myers List, um σ maior não indica uma melhor performance no *lookup*, nesta estrutura a lógica é inversa, ou seja, quanto maior o σ maior será o número de travessias, mas consequentemente usará menos memória.

O novo apontador *shortcut* destas células *skip* é a contribuição principal para resolver o problema da descida dupla e assume um comportamento diferente dependendo da posição da célula na árvore lógica: (i) quando a altura

é diferente de 0, o *shortcut* aponta para a última célula da região que contém a célula acabada de ser criada. (ii) quando a altura é igual a 0, o *shortcut* aponta para `c.jump.jump.jump`, permitindo uma travessia horizontal mais rápida do que antes.

3.2.2 *Lookup*

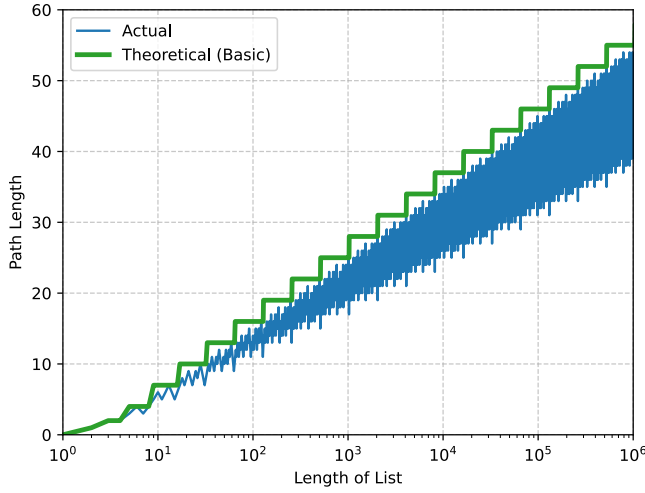
O algoritmo de *lookup* na IARASS mantém-se idêntico à Improved Myers List nas células normais. Nas células *skip*, o algoritmo verifica primeiro se é possível utilizar o *shortcut* para efetuar um salto maior, caso o *shortcut* ultrapasse a posição desejada, o algoritmo avalia o apontador *jump* e caso este também ultrapasse a posição desejada, optamos pela última opção de seguir pelo apontador *next*.

4 Resultados

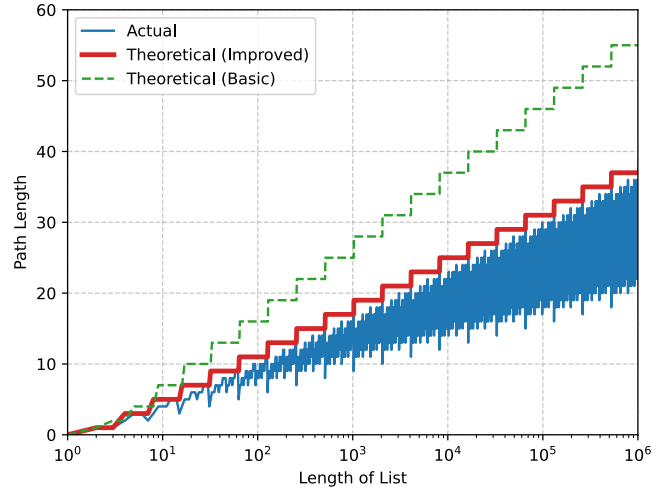
Nesta secção, apresentaremos os resultados obtidos na avaliação das cinco estruturas abordadas neste relatório, Basic, Improved e Advanced Myers List, Dense Advanced Myers List e IARASS. Nesta secção, ainda incluímos os resultados para a estrutura de dados Okasaki List [2].

4.1 Número de Travessias

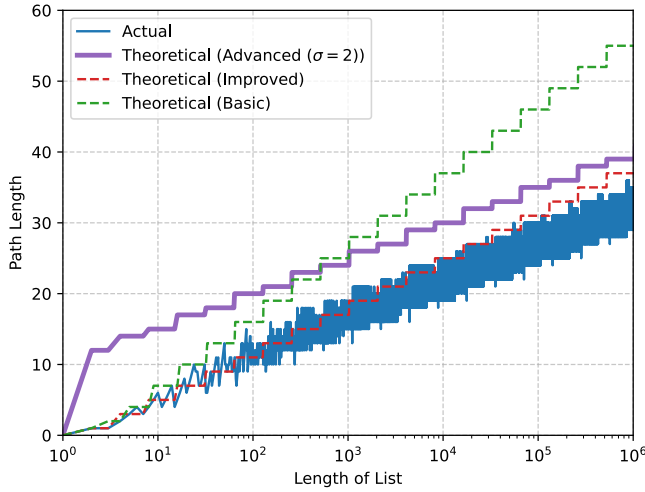
Os gráficos das Figuras 11 e 12 comparam o número de travessias do pior caso com o limite teórico da estrutura, para listas de comprimento entre 1 e 1.000.000 elementos. O teste foi efetuado criando listas de diferentes tamanhos, de 1 elemento até 1.000.000 de elementos, e efetuando um *lookup* sobre todos os elementos, registando o maior número de travessias efetuadas.



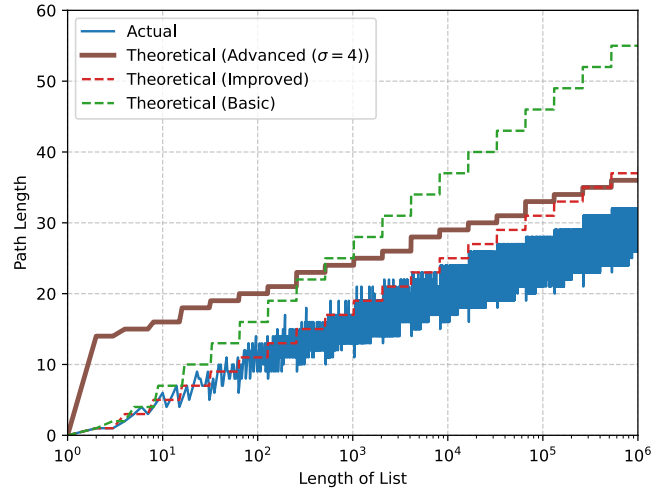
(a) Basic Myers List



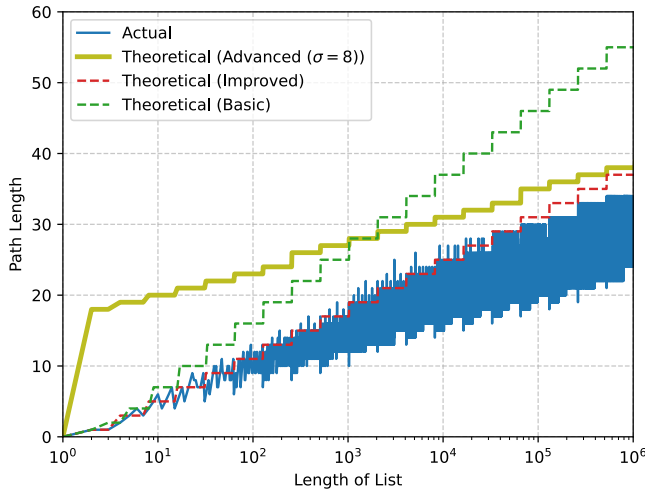
(b) Improved Myers List



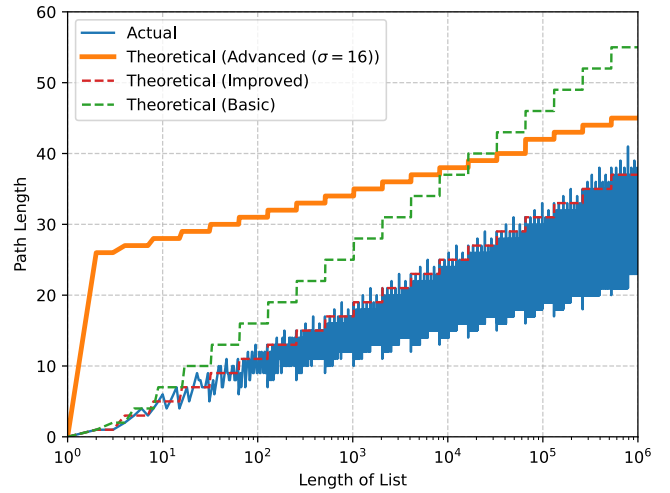
(c) Advanced Myers List ($\sigma = 2$)



(d) Advanced Myers List ($\sigma = 4$)



(e) Advanced Myers List ($\sigma = 8$)



(f) Advanced Myers List ($\sigma = 16$)

Figura 11: Número de travessias - Basic, Improved e Advanced Myers List

Os resultados confirmam a hierarquia Basic $>$ Improved $\geq$ Advanced, mas mostram também que a escolha de um σ tem um impacto direto no ponto a partir do qual a Advanced Myers List compensa mais face à Improved.

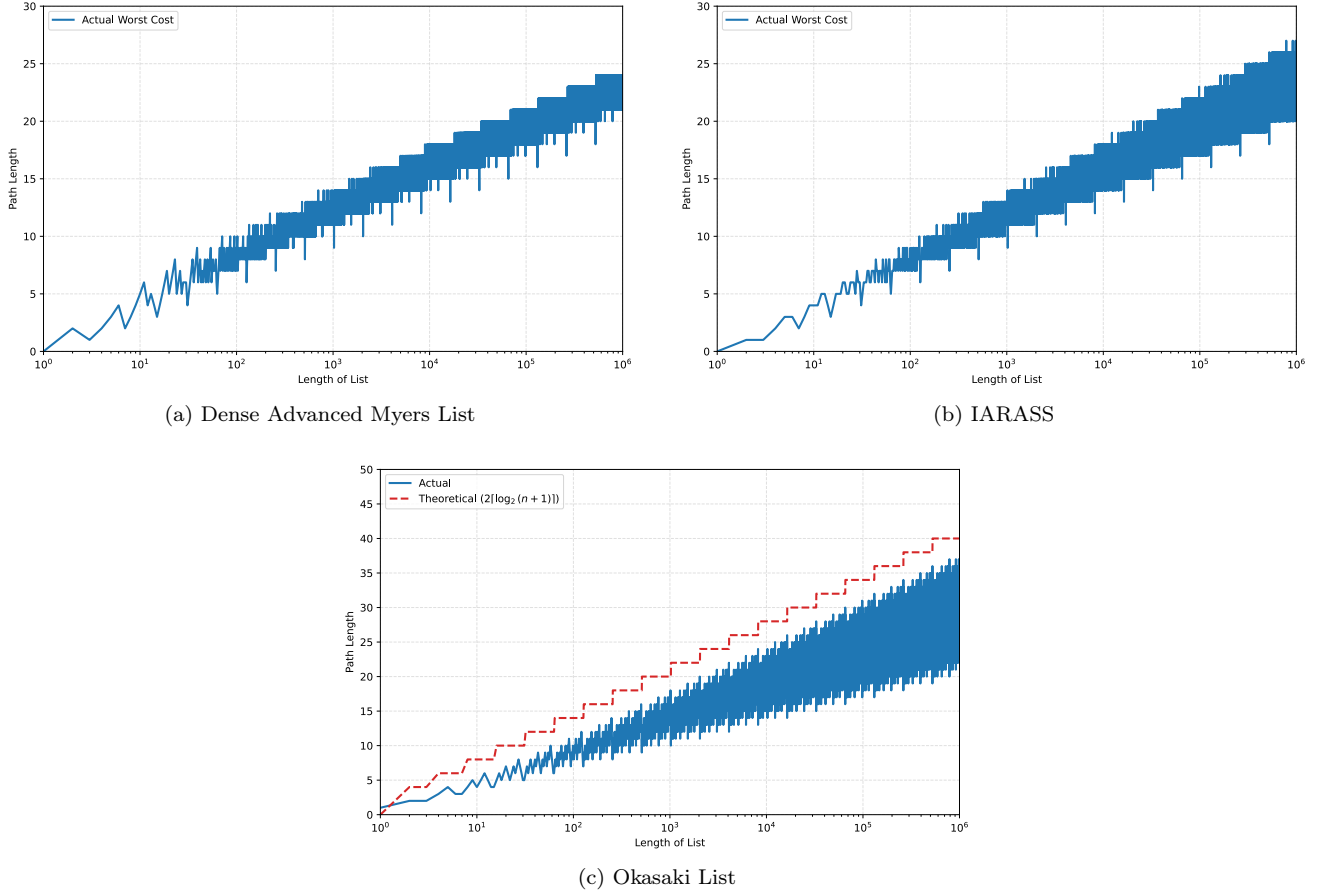


Figura 12: Número de travessias - Dense Advanced Myers List, IARASS e Okasaki List

A Tabela 1 apresenta o número máximo de travessias observado para $n = 1.000.000$, valores retirados da leitura dos gráficos anteriores com valores exatos para comparação direta entre estruturas.

Variante	Pior caso registrado
Basic Myers List	54
Improved Myers List	36
Advanced Myers List ($\sigma = 2$)	36
Advanced Myers List ($\sigma = 4$)	32
Advanced Myers List ($\sigma = 8$)	34
Advanced Myers List ($\sigma = 16$)	41
Dense Advanced Myers List ($\sigma = 2$)	24
IARASS	27
Okasaki List	37

Tabela 1: Pior caso de travessias registrado ($n = 1.000.000$)

4.2 Tempo de Execução

A Tabela 2 apresenta o tempo total de execução de *lookups* sobre todos os elementos de uma lista com 1.000.000 de elementos para todas as variantes. O teste calcula o tempo total necessário para pesquisar todos os elementos de uma lista com 1.000.000 de elementos, ou seja, é efetuado `get 0`, `get 1`, `...` `get 999999`, estando apresentado na tabela o tempo total. O tempo de construção da lista não está incluído.

Variante	Lookup All 1.000.000 (ms)
Basic Myers List	57,42
Improved Myers List	77,46
Advanced Myers List ($\sigma = 2$)	700,22
Advanced Myers List ($\sigma = 4$)	637,29
Advanced Myers List ($\sigma = 8$)	362,93
Advanced Myers List ($\sigma = 16$)	588,77
Dense Advanced Myers List ($\sigma = 2$)	60,35
IARASS	65,00
Okasaki List	51,04

Tabela 2: Tempo de execução

4.3 Uso de Memória

A Tabela 3 apresenta o uso de memória de cada variante para uma lista de 1.000.000 de elementos. Como era esperado, a Basic e a Improved Myers List apresentam exatamente o mesmo uso de memória (38,15 MB), uma vez que ambas mantêm o mesmo número de apontadores por célula, sendo apenas diferentes na maneira como o apontador `jump` é atribuído durante o *cons*. A Okasaki List apresenta o menor uso de memória de todas as variantes (22,89 MB).

A Advanced Myers List (53,41 MB) apresenta um uso de memória superior ao das variantes Basic e Improved não por introduzir novos apontadores, a estrutura de célula mantém-se igual, reaproveitando o próprio apontador `jump` para servir de `uncle/inc/greater` consoante o tipo de nó lógico, mas por cada célula ter de guardar explicitamente a informação de RHD e RED necessária para a sua construção e para a otimização do *lookup* face as variantes anteriores.

A Dense Advanced Myers List (75,02 MB) leva esta ideia mais longe, ao consolidar cada nó lógico numa única célula, os campos `rh`, `red` e o identificador do tipo de nó (o campo `more`, com os construtores `Normal`, `Skip` e `Leaf`) passam a estar presentes em todas as células, que consequentemente exige um consumo de memória ainda superior ao da Advanced original.

Variante	Memória 1.000.000 elementos (MB)
Basic Myers List	38,15
Improved Myers List	38,15
Advanced Myers List	53,41
Dense Advanced Myers List ($\sigma = 2$)	75,02
IARASS	50,86
Okasaki List	22,89

Tabela 3: Uso de memória para cada variante

5 Discussão

A análise e avaliação das estruturas de dados desenvolvidas permitiu chegar a conclusões claras face as perguntas propostas nos objetivos. Relativamente à primeira pergunta, a implementação da Dense Advanced Myers List concluiu que não foi possível obter uma operação *cons* em tempo constante. Esta limitação deve-se à impossibilidade de aceder às células, que são utilizadas como ponteiro `uncle`, `greater` ou `inc`, sem a necessidade de atravessar a lista ou fazer um *lookup*. No entanto, foi possível implementar uma operação *cons* funcional para esta estrutura, com uma complexidade temporal de [INSERIR A COMPLEXIDADE]. Importa destacar que o objetivo de um *cons* com tempo constante exige uma reestruturação da arquitetura, objetivo para futuras investigações.

Apesar de o *cons* em $\mathcal{O}(1)$ não ter sido obtido, a Dense Advanced Myers List apresentou resultados eficazes na leitura, tendo um tempo de execução de *lookup* total de 60,35 ms, valor muito inferior à variante Advanced Myers List com menor valor registado de 362,93 ms. Além disso, ainda apresentou uma melhoria no número de travessias, obtendo o melhor desempenho no pior caso quando comparada a todas as outras estruturas. Este ganho de performance tem um custo direto associado à memória exigindo 75,02 MB para um milhão de elementos, sendo a estrutura mais pesada do nosso estudo. Assim, conclui-se que esta estrutura é ideal para cenários onde a travessia de apontadores é o principal obstáculo.

Relativamente à segunda pergunta, a implementação da IARASS revelou ser um sucesso na diminuição do problema da descida dupla. Ao introduzir células *skip* com um apontador adicional, conseguimos reduzir o número de travessias no pior caso de 36 (variante Improved) para 27. Além disso, apresentou um tempo de execução de 65,00 ms, ultrapassando a Improved Myers List (77,46 ms) e a Advanced Myers List ($\sigma = 8$, 362,93 ms). No que toca a uso de memória, a IARASS utiliza 50,86 MB, valor superior aos 38,15 MB da variante Improved e consumindo menos memória do que a Advanced Myers List (53,41 MB). Assim, conclui-se que esta estrutura apresenta um melhor desempenho global superior ao da Advanced Myers List, otimizando simultaneamente a velocidade de execução, o número de travessias e a eficiência de memória.

Estas duas novas estruturas de dados criadas, quando comparadas com a Okasaki List apresentam um ótimo desempenho quando a prioridade é um menor número de travessias, no entanto são muito inferiores quando a prioridade é o menor uso de memória ou a velocidade de *lookup* quando o objetivo é uma maior eficiência de tempo. Adicionalmente, a Dense Myers e a IARASS apresentam uma vantagem estrutural quando comparadas com a Okasaki List, sendo esta a capacidade de suportar *path queries* ou *range queries*.

6 Conclusão

Em suma, a remoção da restrição permitiu-nos implementar duas novas estruturas de dados com propostas de valor distintas. A Dense Advanced Myers List é a melhor escolha quando a velocidade *lookup* é o mais importante e a memória não é um problema a ser considerado, continuando com a operação de *cons*. Por sua vez, a IARASS mostra ser uma estrutura de dados equilibrada que oferece melhorias consideráveis no desempenho de *lookup* com um aumento moderado na memória.

Referências

- [1] Eugene W. Myers. An applicative random-access stack. *Information Processing Letters*, 17(5):241–248, 1983.
- [2] Chris Okasaki. Purely functional random-access lists. In *Proceedings of the seventh international conference on Functional programming languages and computer architecture*, pages 86–95. ACM, 1995.
- [3] Edward Peters, Yong Qi Foo, and Michael D. Adams. Pushing the information-theoretic limits of random access lists. 2025.